```
!pip install rdkit
```

```
Collecting rdkit
  Downloading rdkit-2026.3.5-cp312-cp312-manylinux_2_28_x86_64.whl.metadata (3.8 kB)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from rdkit) (2.0.2)
Requirement already satisfied: Pillow in /usr/local/lib/python3.12/dist-packages (from rdkit) (11.3.0)
Downloading rdkit-2026.3.5-cp312-cp312-manylinux_2_28_x86_64.whl (37.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 37.4/37.4 MB 36.3 MB/s eta 0:00:00
Installing collected packages: rdkit
Successfully installed rdkit-2026.3.5
```

```
import pandas as pd
import numpy as np

from rdkit import Chem
from rdkit.Chem import Descriptors, Lipinski
```

```
def molecular_features(smiles):

    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return None

    return {
        "molecular_weight": Descriptors.MolWt(mol),
        "logp": Descriptors.MolLogP(mol),
        "h_bond_donors": Lipinski.NumHDonors(mol),
        "h_bond_acceptors": Lipinski.NumHAcceptors(mol),
        "rotatable_bonds": Lipinski.NumRotatableBonds(mol),
        "tpsa": Descriptors.TPSA(mol),
        "num_atoms": mol.GetNumAtoms(),
        "num_rings": Lipinski.RingCount(mol)
    }
```

```
smiles = "CCO"

features = molecular_features(smiles)
```

```
features
```

```
{'molecular_weight': 46.069,
 'logp': -0.0014000000000000123,
 'h_bond_donors': 1,
 'h_bond_acceptors': 1,
 'rotatable_bonds': 0,
 'tpsa': 20.23,
 'num_atoms': 3,
 'num_rings': 0}
```

```
molecules = {
    "Ethanol": "CCO",
    "Aspirin": "CC(=O)OC1=CC=CC=C1C(=O)O",
    "Caffeine": "Cn1c(=O)c2c(ncn2C)n(C)c1=O"
}

rows = []

for name, smiles in molecules.items():

    features = molecular_features(smiles)

    if features:
        features["name"] = name
        features["smiles"] = smiles
        rows.append(features)

df = pd.DataFrame(rows)

df
```

	molecular_weight	logp	h_bond_donors	h_bond_acceptors	rotatable_bonds	tpsa	num_atoms	num_rings	nam
0	46.069	-0.0014	1	1	0	20.23	3	0	Ethan
1	180.159	1.3101	1	3	2	63.60	13	1	Aspir
2	194.194	-1.0293	0	3	0	61.82	14	2	Caffeir

Next steps: Generate code with df | New interactive sheet

```
import pandas as pd

url = "https://raw.githubusercontent.com/deepchem/deepchem/master/datasets/delaney-processed.csv"

df = pd.read_csv(url)

print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

df.head()
```

```
Shape: (1128, 10)

Columns:
['Compound ID', 'ESOL predicted log solubility in mols per litre', 'Minimum Degree', 'Molecular Weight', 'Numb
```

	Compound ID	ESOL predicted log solubility in mols per litre	Minimum Degree	Molecular Weight	Number of H-Bond Donors	Number of Rings	Number of Rotatable Bonds	Polar Surface Area	measured log solubility in mols per litre	
0	Amigdalin	-0.974	1	457.432	7	3	7	202.32	-0.77	OCC3OC(OCC2OC(OC(C#
1	Fenfuram	-2.885	1	201.225	1	2	2	42.24	-3.30	
2	citral	-2.579	1	152.237	0	0	4	17.07	-2.06	
3	Picene	-6.618	2	278.354	0	5	0	0.00	-7.87	c1ccc2
4	Thiophene	-2.232	2	84.143	0	1	0	0.00	-1.33	

Next steps: Generate code with df | New interactive sheet

```
df = df[
    [
        "Compound ID",
        "smiles",
        "measured log solubility in mols per litre"
    ]
].copy()

df.head()
```

	Compound ID	smiles	measured log solubility in mols per litre
0	Amigdalin	OCC3OC(OCC2OC(OC(C#N)c1ccccc1)C(O)C(O)C2O)C(O)...	-0.77
1	Fenfuram	Cc1occc1C(=O)Nc2ccccc2	-3.30
2	citral	CC(C)=CCCC(C)=CC(=O)	-2.06
3	Picene	c1ccc2c(c1)ccc3c2ccc4c5ccccc5ccc43	-7.87
4	Thiophene	c1ccsc1	-1.33

Next steps: Generate code with df | New interactive sheet

```
print("Number of molecules:", len(df))

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate SMILES:")
print(df["smiles"].duplicated().sum())

print("\nTarget statistics:")
print(df["measured log solubility in mols per litre"].describe())
```

```
Number of molecules: 1128

Missing values:
Compound ID                                 0
smiles                                      0
measured log solubility in mols per litre    0
dtype: int64

Duplicate SMILES:
0

Target statistics:
count    1128.000000
mean       -3.050102
std         2.096441
min       -11.600000
25%        -4.317500
50%        -2.860000
75%        -1.600000
max         1.580000
Name: measured log solubility in mols per litre, dtype: float64
```

```
from rdkit import Chem

def is_valid_smiles(smiles):
    return Chem.MolFromSmiles(smiles) is not None

df["valid_smiles"] = df["smiles"].apply(is_valid_smiles)

print(df["valid_smiles"].value_counts())
```

```
valid_smiles
True    1128
Name: count, dtype: int64
```

```
df = df[df["valid_smiles"]].drop(columns=["valid_smiles"])

df = df.drop_duplicates(subset="smiles").reset_index(drop=True)
```

```
print("Final dataset shape:", df.shape)
```

```
Final dataset shape: (1128, 3)
```

```
from rdkit.Chem import Descriptors, Lipinski

def molecular_features(smiles):

    mol = Chem.MolFromSmiles(smiles)

    return [
        Descriptors.MolWt(mol),
        Descriptors.MolLogP(mol),
        Lipinski.NumHDonors(mol),
        Lipinski.NumHAcceptors(mol),
        Lipinski.NumRotatableBonds(mol),
        Descriptors.TPSA(mol),
        mol.GetNumAtoms(),
        Lipinski.RingCount(mol)
    ]

feature_names = [
    "molecular_weight",
    "logp",
    "h_bond_donors",
    "h_bond_acceptors",
    "rotatable_bonds",
    "tpsa",
    "num_atoms",
    "num_rings"
]

features = df["smiles"].apply(molecular_features)

X = pd.DataFrame(
    features.tolist(),
    columns=feature_names
)

y = df["measured log solubility in mols per litre"]

print("X shape:", X.shape)
```

```
print("y shape:", y.shape)

X.head()
```

```
X shape: (1128, 8)
y shape: (1128,)
```

	molecular_weight	logp	h_bond_donors	h_bond_acceptors	rotatable_bonds	tpsa	num_atoms	num_rings
0	457.432	-3.10802	7	12	7	202.32	32	3
1	201.225	2.84032	1	2	2	42.24	15	2
2	152.237	2.87800	0	1	4	17.07	11	0
3	278.354	6.29940	0	0	0	0.00	22	5
4	84.143	1.74810	0	1	0	0.00	5	1

Next steps: Generate code with X | New interactive sheet

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training:", X_train.shape)
print("Testing :", X_test.shape)
```

```
Training: (902, 8)
Testing : (226, 8)
```

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=300,
```

```
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)
```

```
RandomForestRegressor(n_estimators=300, n_jobs=-1, random_state=42)
```

```
y_pred = rf_model.predict(X_test)
```

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(
    y_test,
    y_pred
) ** 0.5
r2 = r2_score(y_test, y_pred)

print("Random Forest Results")
print("---------------------")
print(f"MAE  : {mae:.4f}")
print(f"RMSE : {rmse:.4f}")
print(f"R²   : {r2:.4f}")
```

```
Random Forest Results
---------------------
MAE  : 0.5563
RMSE : 0.8053
R²   : 0.8628
```

```
from sklearn.ensemble import GradientBoostingRegressor

gb_model = GradientBoostingRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=3,
    random_state=42
)

gb_model.fit(X_train, y_train)

gb_pred = gb_model.predict(X_test)

gb_mae = mean_absolute_error(y_test, gb_pred)
gb_rmse = mean_squared_error(y_test, gb_pred) ** 0.5
gb_r2 = r2_score(y_test, gb_pred)

print("Gradient Boosting Results")
print("-------------------------")
print(f"MAE  : {gb_mae:.4f}")
print(f"RMSE : {gb_rmse:.4f}")
print(f"R²   : {gb_r2:.4f}")
```

```
Gradient Boosting Results
-------------------------
MAE  : 0.5631
RMSE : 0.7935
R²   : 0.8668
```

```
results = pd.DataFrame({
    "Model": [
        "Random Forest",
        "Gradient Boosting"
    ],
    "MAE": [
        mae,
        gb_mae
    ],
    "RMSE": [
        rmse,
        gb_rmse
    ],
```

```
    "R2": [
        r2,
        gb_r2
    ]
})

results.sort_values("R2", ascending=False)
```

	Model	MAE	RMSE	R2
1	Gradient Boosting	0.563137	0.793457	0.866808
0	Random Forest	0.556308	0.805284	0.862807

```
from rdkit.Chem import AllChem
from rdkit import DataStructs

def get_morgan_fingerprint(smiles, radius=2, n_bits=2048):
    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return None

    fingerprint = AllChem.GetMorganFingerprintAsBitVect(
        mol,
        radius,
        nBits=n_bits
    )

    return np.array(fingerprint)

fingerprints = df["smiles"].apply(get_morgan_fingerprint)

X_fp = np.array(fingerprints.tolist())

print("Fingerprint shape:", X_fp.shape)
```

```
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
[01:13:17] DEPRECATION WARNING: please use MorganGenerator
```

```
X_fp_train, X_fp_test, y_fp_train, y_fp_test = train_test_split(
    X_fp,
    y,
    test_size=0.20,
    random_state=42
)

print(X_fp_train.shape)
print(X_fp_test.shape)
```

```
(902, 2048)
(226, 2048)
```

```
rf_fp = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)

rf_fp.fit(X_fp_train, y_fp_train)

y_fp_pred = rf_fp.predict(X_fp_test)
```

```
rf_fp = RandomForestRegressor(
    n_estimators=300,
```

```
    random_state=42,
    n_jobs=-1
)

rf_fp.fit(X_fp_train, y_fp_train)

y_fp_pred = rf_fp.predict(X_fp_test)
```

```
fp_mae = mean_absolute_error(y_fp_test, y_fp_pred)

fp_rmse = mean_squared_error(
    y_fp_test,
    y_fp_pred
) ** 0.5

fp_r2 = r2_score(
    y_fp_test,
    y_fp_pred
)

print("Morgan Fingerprint + Random Forest")
print("-----------------------------------")
print(f"MAE  : {fp_mae:.4f}")
print(f"RMSE : {fp_rmse:.4f}")
print(f"R²   : {fp_r2:.4f}")
```

```
Morgan Fingerprint + Random Forest
-----------------------------------
MAE  : 0.8747
RMSE : 1.1588
R²   : 0.7159
```

```
comparison = pd.DataFrame({
    "Approach": [
        "RDKit Descriptors + Random Forest",
        "RDKit Descriptors + Gradient Boosting",
        "Morgan Fingerprints + Random Forest"
    ],
    "MAE": [
        mae,
        gb_mae,
        fp_mae
```

```
    ],
    "RMSE": [
        rmse,
        gb_rmse,
        fp_rmse
    ],
    "R2": [
        r2,
        gb_r2,
        fp_r2
    ]
})

comparison.sort_values(
    "R2",
    ascending=False
)
```

	Approach	MAE	RMSE	R2
1	RDKit Descriptors + Gradient Boosting	0.563137	0.793457	0.866808
0	RDKit Descriptors + Random Forest	0.556308	0.805284	0.862807
2	Morgan Fingerprints + Random Forest	0.874709	1.158799	0.715915

```
!pip install -q xgboost
```

```
from xgboost import XGBRegressor
```

```
xgb_model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xgb_model.fit(
    X_train,
    y_train
)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.03, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
             max_leaves=None, min_child_weight=2, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=500,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
xgb_pred = xgb_model.predict(X_test)
```

```
xgb_mae = mean_absolute_error(y_test, xgb_pred)

xgb_rmse = mean_squared_error(
    y_test,
    xgb_pred
) ** 0.5
```

```
xgb_r2 = r2_score(
    y_test,
    xgb_pred
)

print("XGBoost Results")
print("----------------")
print(f"MAE  : {xgb_mae:.4f}")
print(f"RMSE : {xgb_rmse:.4f}")
print(f"R²   : {xgb_r2:.4f}")
```

```
XGBoost Results
----------------
MAE  : 0.5264
RMSE : 0.7359
R²   : 0.8854
```

```
final_comparison = pd.DataFrame({
    "Model": [
        "Random Forest",
        "Gradient Boosting",
        "Morgan + Random Forest",
        "XGBoost"
    ],
    "MAE": [
        mae,
        gb_mae,
        fp_mae,
        xgb_mae
    ],
    "RMSE": [
        rmse,
        gb_rmse,
        fp_rmse,
        xgb_rmse
    ],
    "R2": [
        r2,
        gb_r2,
        fp_r2,
        xgb_r2
    ]
```

```
})

final_comparison.sort_values(
    "R2",
    ascending=False
).reset_index(drop=True)
```

	Model	MAE	RMSE	R2
0	XGBoost	0.526429	0.735947	0.885416
1	Gradient Boosting	0.563137	0.793457	0.866808
2	Random Forest	0.556308	0.805284	0.862807
3	Morgan + Random Forest	0.874709	1.158799	0.715915

```
from rdkit.Chem.Scaffolds import MurckoScaffold

def get_scaffold(smiles):
    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return None

    return MurckoScaffold.MurckoScaffoldSmiles(mol=mol)

df["scaffold"] = df["smiles"].apply(get_scaffold)

print(df[["smiles", "scaffold"]].head())
```

```
                                              smiles  \
0  OCC3OC(OCC2OC(OC(C#N)c1ccccc1)C(O)C(O)C2O)C(O)...
1                            Cc1occc1C(=O)Nc2ccccc2
2                              CC(C)=CCCC(C)=CC(=O)
```

```
3                 c1ccc2c(c1)ccc3c2ccc4c5ccccc5ccc43
4                                          c1ccsc1

                             scaffold
0    c1ccc(COC2CCCC(COC3CCCCO3)O2)cc1
1               O=C(Nc1ccccc1)c1ccoc1
2
3  c1ccc2c(c1)ccc1c2ccc2c3ccccc3ccc21
4                            c1ccsc1
```

```
scaffold_groups = (
    df.groupby("scaffold")
      .indices
)

print("Number of unique scaffolds:", len(scaffold_groups))
```

```
Number of unique scaffolds: 269
```

```
import numpy as np

rng = np.random.RandomState(42)

scaffolds = list(scaffold_groups.keys())
rng.shuffle(scaffolds)

train_indices = []
test_indices = []

target_train_size = int(0.80 * len(df))

for scaffold in scaffolds:

    indices = list(scaffold_groups[scaffold])

    if len(train_indices) + len(indices) <= target_train_size:
        train_indices.extend(indices)
    else:
        test_indices.extend(indices)

train_indices = np.array(train_indices)
test_indices = np.array(test_indices)
```

```
print("Train molecules:", len(train_indices))
print("Test molecules :", len(test_indices))
```

```
Train molecules: 902
Test molecules : 226
```

```
X_scaffold_train = X.iloc[train_indices]
X_scaffold_test = X.iloc[test_indices]

y_scaffold_train = y.iloc[train_indices]
y_scaffold_test = y.iloc[test_indices]

print(X_scaffold_train.shape)
print(X_scaffold_test.shape)
```

```
(902, 8)
(226, 8)
```

```
xgb_scaffold = XGBRegressor(
    n_estimators=500,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xgb_scaffold.fit(
    X_scaffold_train,
    y_scaffold_train
)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.8, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.03, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
             max_leaves=None, min_child_weight=2, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=500,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
scaffold_pred = xgb_scaffold.predict(
    X_scaffold_test
)

scaffold_mae = mean_absolute_error(
    y_scaffold_test,
    scaffold_pred
)

scaffold_rmse = mean_squared_error(
    y_scaffold_test,
    scaffold_pred
) ** 0.5

scaffold_r2 = r2_score(
    y_scaffold_test,
    scaffold_pred
)

print("Scaffold Split — XGBoost")
print("------------------------")
print(f"MAE  : {scaffold_mae:.4f}")
print(f"RMSE : {scaffold_rmse:.4f}")
print(f"R²   : {scaffold_r2:.4f}")
```

```
Scaffold Split — XGBoost
------------------------
MAE  : 0.7025
RMSE : 0.9322
R²   : 0.8634
```

```
X_combined = np.hstack([
    X.values,
    X_fp
])

print("Combined feature shape:", X_combined.shape)
```

```
Combined feature shape: (1128, 2056)
```

```
X_comb_train, X_comb_test, y_comb_train, y_comb_test = train_test_split(
    X_combined,
    y,
    test_size=0.20,
    random_state=42
)

print("Train:", X_comb_train.shape)
print("Test :", X_comb_test.shape)
```

```
Train: (902, 2056)
Test : (226, 2056)
```

```
xgb_combined = XGBRegressor(
    n_estimators=600,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.7,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)
```

```
xgb_combined.fit(
    X_comb_train,
    y_comb_train
)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.7, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.03, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
             max_leaves=None, min_child_weight=2, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=600,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
combined_pred = xgb_combined.predict(
    X_comb_test
)

combined_mae = mean_absolute_error(
    y_comb_test,
    combined_pred
)

combined_rmse = mean_squared_error(
    y_comb_test,
    combined_pred
) ** 0.5

combined_r2 = r2_score(
    y_comb_test,
    combined_pred
)

print("XGBoost + Descriptors + Morgan")
print("-------------------------------")
```

```
print(f"MAE  : {combined_mae:.4f}")
print(f"RMSE : {combined_rmse:.4f}")
print(f"R²   : {combined_r2:.4f}")
```

```
XGBoost + Descriptors + Morgan
--------------------------------
MAE  : 0.4832
RMSE : 0.6747
R²   : 0.9037
```

```
leaderboard = pd.DataFrame({
    "Model": [
        "Random Forest",
        "Gradient Boosting",
        "Morgan + Random Forest",
        "XGBoost",
        "XGBoost + Descriptors + Morgan"
    ],
    "MAE": [
        mae,
        gb_mae,
        fp_mae,
        xgb_mae,
        combined_mae
    ],
    "RMSE": [
        rmse,
        gb_rmse,
        fp_rmse,
        xgb_rmse,
        combined_rmse
    ],
    "R2": [
        r2,
        gb_r2,
        fp_r2,
        xgb_r2,
        combined_r2
    ]
})

leaderboard.sort_values(
    "R2",
```

```
    ascending=False
).reset_index(drop=True)
```

	Model	MAE	RMSE	R2
0	XGBoost + Descriptors + Morgan	0.483218	0.674749	0.903680
1	XGBoost	0.526429	0.735947	0.885416
2	Gradient Boosting	0.563137	0.793457	0.866808
3	Random Forest	0.556308	0.805284	0.862807
4	Morgan + Random Forest	0.874709	1.158799	0.715915

```
X_scaffold_combined_train = X_combined[train_indices]
X_scaffold_combined_test = X_combined[test_indices]

y_scaffold_combined_train = y.iloc[train_indices]
y_scaffold_combined_test = y.iloc[test_indices]

print("Train:", X_scaffold_combined_train.shape)
print("Test :", X_scaffold_combined_test.shape)
```

```
Train: (902, 2056)
Test : (226, 2056)
```

```
xgb_scaffold_combined = XGBRegressor(
    n_estimators=600,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.7,
```

```
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
)

xgb_scaffold_combined.fit(
    X_scaffold_combined_train,
    y_scaffold_combined_train
)
```

```
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=0.7, device=None, early_stopping_rounds=None,
             enable_categorical=True, eval_metric=None, feature_types=None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=0.03, max_bin=None, max_cat_threshold=None,
             max_cat_to_onehot=None, max_delta_step=None, max_depth=4,
             max_leaves=None, min_child_weight=2, missing=nan,
             monotone_constraints=None, multi_strategy=None, n_estimators=600,
             n_jobs=-1, num_parallel_tree=None, ...)
```

```
scaffold_combined_pred = xgb_scaffold_combined.predict(
    X_scaffold_combined_test
)
```

```
scaffold_combined_mae = mean_absolute_error(
    y_scaffold_combined_test,
    scaffold_combined_pred
)

scaffold_combined_rmse = mean_squared_error(
    y_scaffold_combined_test,
    scaffold_combined_pred
) ** 0.5

scaffold_combined_r2 = r2_score(
```

```
    y_scaffold_combined_test,
    scaffold_combined_pred
)

print("XGBoost + Descriptors + Morgan")
print("Scaffold Split")
print("----------------------------")
print(f"MAE  : {scaffold_combined_mae:.4f}")
print(f"RMSE : {scaffold_combined_rmse:.4f}")
print(f"R²   : {scaffold_combined_r2:.4f}")
```

```
XGBoost + Descriptors + Morgan
Scaffold Split
----------------------------
MAE  : 0.6983
RMSE : 0.9382
R²   : 0.8617
```

```
X_scaffold_train = X_combined[train_indices]
X_scaffold_test = X_combined[test_indices]

y_scaffold_train = y.iloc[train_indices]
y_scaffold_test = y.iloc[test_indices]

print("Training shape:", X_scaffold_train.shape)
print("Testing shape :", X_scaffold_test.shape)
```

```
Training shape: (902, 2056)
Testing shape : (226, 2056)
```

```
from xgboost import XGBRegressor

model_scaffold = XGBRegressor(
    n_estimators=600,
    learning_rate=0.03,
    max_depth=4,
    min_child_weight=2,
    subsample=0.8,
    colsample_bytree=0.7,
    objective="reg:squarederror",
    random_state=42,
    n_jobs=-1
```

```
)

model_scaffold.fit(
    X_scaffold_train,
    y_scaffold_train
)

print("Model training completed.")
```

```
Model training completed.
```

```
scaffold_pred = model_scaffold.predict(
    X_scaffold_test
)
```

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae_scaffold = mean_absolute_error(
    y_scaffold_test,
    scaffold_pred
)

rmse_scaffold = mean_squared_error(
    y_scaffold_test,
    scaffold_pred
) ** 0.5

r2_scaffold = r2_score(
    y_scaffold_test,
    scaffold_pred
)

print("===== Scaffold Split Results =====")
print(f"MAE  : {mae_scaffold:.4f}")
print(f"RMSE : {rmse_scaffold:.4f}")
print(f"R²   : {r2_scaffold:.4f}")
```

```
===== Scaffold Split Results =====
MAE  : 0.6983
RMSE : 0.9382
R²   : 0.8617
```

```
!pip install -q shap
```

```
import shap
import matplotlib.pyplot as plt
```

```
feature_names = [
    "Molecular Weight",
    "LogP",
    "H-Bond Donors",
    "H-Bond Acceptors",
    "Rotatable Bonds",
    "TPSA",
    "Number of Atoms",
    "Number of Rings"
]

feature_names += [
    f"Morgan_{i}"
    for i in range(2048)
]

print("Total features:", len(feature_names))
```

```
Total features: 2056
```

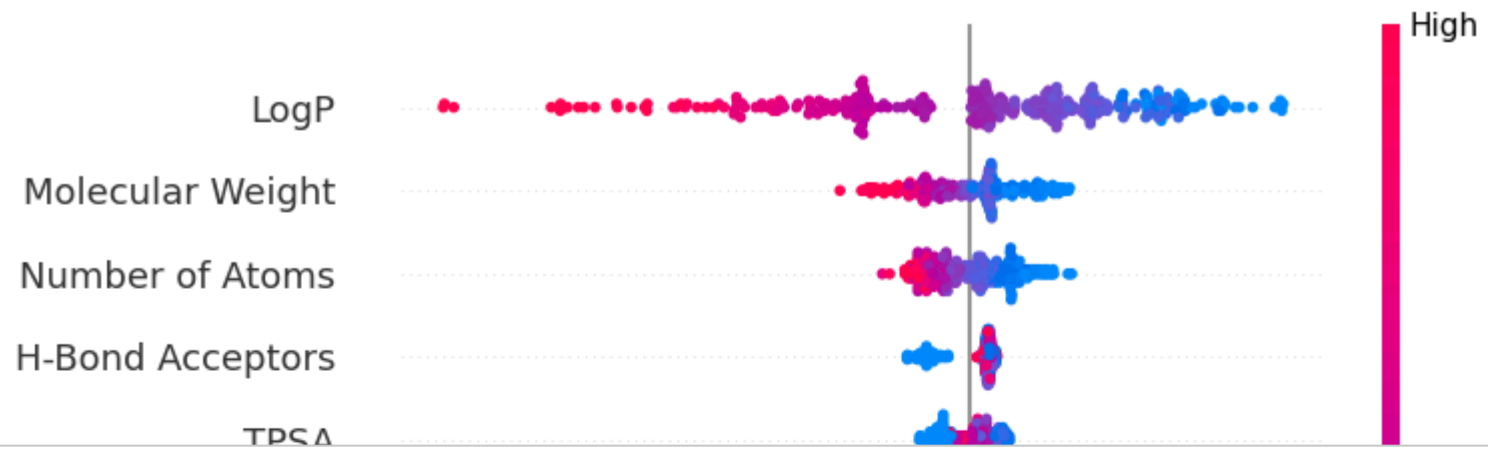
```
explainer = shap.TreeExplainer(
    xgb_combined
)
```

```
X_shap = X_combined[:300]

shap_values = explainer.shap_values(
    X_shap
```

```
)

print("SHAP shape:", shap_values.shape)
```

```
SHAP shape: (300, 2056)
```

```
shap.summary_plot(
    shap_values,
    X_shap,
    feature_names=feature_names,
    max_display=15
)
```




```
from rdkit import Chem
from rdkit.Chem import Descriptors, Lipinski


def analyze_molecule(smiles):

    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return {
            "valid": False,
            "error": "Invalid SMILES"
        }

    molecular_weight = Descriptors.MolWt(mol)
```

```
    logp = Descriptors.MolLogP(mol)
    hbd = Lipinski.NumHDonors(mol)
    hba = Lipinski.NumHAcceptors(mol)
    rotatable = Lipinski.NumRotatableBonds(mol)
    tpsa = Descriptors.TPSA(mol)

    violations = 0

    if molecular_weight > 500:
        violations += 1

    if logp > 5:
        violations += 1

    if hbd > 5:
        violations += 1

    if hba > 10:
        violations += 1

    return {
        "valid": True,
        "molecular_weight": round(molecular_weight, 2),
        "logp": round(logp, 2),
        "h_bond_donors": hbd,
        "h_bond_acceptors": hba,
        "rotatable_bonds": rotatable,
        "tpsa": round(tpsa, 2),
        "lipinski_violations": violations,
        "drug_like": violations <= 1
    }
```

```
aspirin = "CC(=O)OC1=CC=CC=C1C(=O)O"

result = analyze_molecule(aspirin)

result
```

```
{'valid': True,
 'molecular_weight': 180.16,
 'logp': 1.31,
 'h_bond_donors': 1,
 'h_bond_acceptors': 3,
```

```
 'rotatable_bonds': 2,
 'tpsa': 63.6,
 'lipinski_violations': 0,
 'drug_like': True}
```

```
test_molecules = {
    "Ethanol": "CCO",
    "Aspirin": "CC(=O)OC1=CC=CC=C1C(=O)O",
    "Caffeine": "Cn1c(=O)c2c(ncn2C)n(C)c1=O"
}

for name, smiles in test_molecules.items():

    result = analyze_molecule(smiles)

    print("\n", name)
    print("-" * 30)

    for key, value in result.items():
        print(f"{key}: {value}")
```

```
 Ethanol
------------------------------
valid: True
molecular_weight: 46.07
logp: -0.0
h_bond_donors: 1
h_bond_acceptors: 1
rotatable_bonds: 0
tpsa: 20.23
lipinski_violations: 0
drug_like: True

 Aspirin
------------------------------
valid: True
molecular_weight: 180.16
logp: 1.31
h_bond_donors: 1
h_bond_acceptors: 3
```

```
rotatable_bonds: 2
tpsa: 63.6
lipinski_violations: 0
drug_like: True

 Caffeine
------------------------------
valid: True
molecular_weight: 194.19
logp: -1.03
h_bond_donors: 0
h_bond_acceptors: 3
rotatable_bonds: 0
tpsa: 61.82
lipinski_violations: 0
drug_like: True
```

```
from rdkit import DataStructs
from rdkit.Chem import AllChem


def get_fingerprint(smiles):

    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return None

    return AllChem.GetMorganFingerprintAsBitVect(
        mol,
        radius=2,
        nBits=2048
    )
```

```
dataset_fingerprints = []

for smiles in df["smiles"]:

    fp = get_fingerprint(smiles)

    dataset_fingerprints.append(fp)
```

```
print("Fingerprints created:", len(dataset_fingerprints))
```

```
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
[01:26:57] DEPRECATION WARNING: please use MorganGenerator
```

```
def find_similar_molecules(
    query_smiles,
    top_n=10
):

    query_fp = get_fingerprint(query_smiles)

    if query_fp is None:
        return None

    similarities = []

    for i, fp in enumerate(dataset_fingerprints):

        similarity = DataStructs.TanimotoSimilarity(
            query_fp,
            fp
        )

        similarities.append({
            "index": i,
            "similarity": similarity
```

```
            })

    similarities = sorted(
        similarities,
        key=lambda x: x["similarity"],
        reverse=True
    )

    results = []

    for item in similarities[:top_n]:

        idx = item["index"]

        results.append({
            "compound_id": df.iloc[idx]["Compound ID"],
            "smiles": df.iloc[idx]["smiles"],
            "similarity": round(
                item["similarity"],
                4
            ),
            "measured_logS": df.iloc[idx][
                "measured log solubility in mols per litre"
            ]
        })

    return pd.DataFrame(results)
```

```
aspirin = "CC(=O)OC1=CC=CC=C1C(=O)O"

similar_molecules = find_similar_molecules(
    aspirin,
    top_n=10
)

similar_molecules
```

```
[01:27:28] DEPRECATION WARNING: please use MorganGenerator
```

	compound_id	smiles	similarity	measured_logS
0	Dimethyl phthalate	COC(=O)c1ccccc1C(=O)OC	0.4000	-1.660
1	Acetophenone	CC(=O)c1ccccc1	0.3793	-1.280
2	Isoprocarb	CNC(=O)Oc1ccccc1C(C)C	0.3684	-2.863
3	Diethyl phthalate	CCOC(=O)c1ccccc1C(=O)OCC	0.3636	-2.350
4	Benzophenone	O=C(c1ccccc1)c2ccccc2	0.3571	-3.120
5	Metolcarb	c1ccccc1(OC(=O)NC)	0.3529	-1.803
6	Methyl benzoate	COC(=O)c1ccccc1	0.3438	-1.850
7	Acetanilide	CC(=O)Nc1ccccc1	0.3438	-1.330
8	Propoxur	CNC(=O)Oc1ccccc1OC(C)C	0.3333	-2.050
9	Carbaryl	CNC(=O)Oc1cccc2ccccc12	0.3333	-3.224

Next steps: Generate code with `similar_molecules` | New interactive sheet

```
def prepare_molecule_features(smiles):

    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return None

    # RDKit descriptors
    descriptors = [
        Descriptors.MolWt(mol),
        Descriptors.MolLogP(mol),
        Lipinski.NumHDonors(mol),
```

```
            Lipinski.NumHAcceptors(mol),
            Lipinski.NumRotatableBonds(mol),
            Descriptors.TPSA(mol),
            mol.GetNumAtoms(),
            Lipinski.RingCount(mol)
        ]

        # Morgan fingerprint
        fingerprint = AllChem.GetMorganFingerprintAsBitVect(
            mol,
            radius=2,
            nBits=2048
        )

        fingerprint_array = np.array(fingerprint)

        # Combine
        combined = np.concatenate([
            descriptors,
            fingerprint_array
        ])

        return combined
```

```
    def predict_solubility(smiles):

        features = prepare_molecule_features(smiles)

        if features is None:
            return None

        features = features.reshape(1, -1)

        prediction = xgb_combined.predict(
            features
        )[0]

        return float(prediction)
```

```
    aspirin = "CC(=O)OC1=CC=CC=C1C(=O)O"

    prediction = predict_solubility(
```

```
    aspirin
)

print("Predicted LogS:", prediction)
```

```
Predicted LogS: -2.0009090900421143
[01:28:31] DEPRECATION WARNING: please use MorganGenerator
```

```
def virtual_screening(
    molecules,
    top_n=20
):

    results = []

    for molecule_id, smiles in molecules.items():

        features = prepare_molecule_features(
            smiles
        )

        if features is None:
            continue

        prediction = xgb_combined.predict(
            features.reshape(1, -1)
        )[0]

        mol = Chem.MolFromSmiles(smiles)

        results.append({
            "molecule_id": molecule_id,
            "smiles": smiles,
            "predicted_logS": float(prediction),
            "molecular_weight": Descriptors.MolWt(mol),
            "logp": Descriptors.MolLogP(mol),
            "tpsa": Descriptors.TPSA(mol),
            "hbd": Lipinski.NumHDonors(mol),
            "hba": Lipinski.NumHAcceptors(mol)
        })

    results = pd.DataFrame(results)
```

```
    # Higher LogS = more soluble
    results = results.sort_values(
        "predicted_logS",
        ascending=False
    )

    return results.head(top_n).reset_index(
        drop=True
    )
```

```
candidate_molecules = {

    "Ethanol":
        "CCO",

    "Aspirin":
        "CC(=O)OC1=CC=CC=C1C(=O)O",

    "Caffeine":
        "Cn1c(=O)c2c(ncn2C)n(C)c1=O",

    "Paracetamol":
        "CC(=O)NC1=CC=C(O)C=C1",

    "Ibuprofen":
        "CC(C)CC1=CC=C(C=C1)[C@@H](C)C(=O)O",

    "Acetaminophen":
        "CC(=O)NC1=CC=C(O)C=C1"
}
```

```
screened = virtual_screening(
    candidate_molecules,
    top_n=10
)

screened
```

```
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
[01:29:14] DEPRECATION WARNING: please use MorganGenerator
```

	molecule_id	smiles	predicted_logS	molecular_weight	logp	tpsa	hbd	hba
0	Ethanol	CCO	0.902729	46.069	-0.0014	20.23	1	1
1	Caffeine	Cn1c(=O)c2c(ncn2C)n(C)c1=O	-1.007632	194.194	-1.0293	61.82	0	3
2	Acetaminophen	CC(=O)NC1=CC=C(O)C=C1	-1.686459	151.165	1.3506	49.33	2	2
3	Paracetamol	CC(=O)NC1=CC=C(O)C=C1	-1.686459	151.165	1.3506	49.33	2	2
4	Aspirin	CC(=O)OC1=CC=CC=C1C(=O)O	-2.000909	180.159	1.3101	63.60	1	3
5	Ibuprofen	CC(C)CC1=CC=C(C=C1)[C@@H](C)C(=O)O	-3.403788	206.285	3.0732	37.30	1	1

Next steps: Generate code with screened | New interactive sheet

```
def molecule_analyzer(smiles, top_n=5):

    # ----------------------------------------
    # 1. Validate molecule
    # ----------------------------------------

    mol = Chem.MolFromSmiles(smiles)

    if mol is None:
        return {
            "valid": False,
            "error": "Invalid SMILES"
        }

    # ----------------------------------------
    # 2. Molecular properties
```

```
# ----------------------------------------

molecular_weight = Descriptors.MolWt(mol)
logp = Descriptors.MolLogP(mol)
hbd = Lipinski.NumHDonors(mol)
hba = Lipinski.NumHAcceptors(mol)
rotatable = Lipinski.NumRotatableBonds(mol)
tpsa = Descriptors.TPSA(mol)
atoms = mol.GetNumAtoms()
rings = Lipinski.RingCount(mol)

# ----------------------------------------
# 3. Lipinski violations
# ----------------------------------------

violations = 0

if molecular_weight > 500:
    violations += 1

if logp > 5:
    violations += 1

if hbd > 5:
    violations += 1

if hba > 10:
    violations += 1

# ----------------------------------------
# 4. ML prediction
# ----------------------------------------

features = prepare_molecule_features(smiles)

prediction = xgb_combined.predict(
    features.reshape(1, -1)
)[0]

# ----------------------------------------
# 5. Similarity search
# ----------------------------------------

similar = find_similar_molecules(
```

```
            smiles,
            top_n=top_n
        )

        # ----------------------------------------
        # 6. Final result
        # ----------------------------------------

        return {
            "valid": True,

            "molecular_properties": {
                "molecular_weight": round(
                    molecular_weight, 2
                ),
                "logp": round(
                    logp, 2
                ),
                "h_bond_donors": hbd,
                "h_bond_acceptors": hba,
                "rotatable_bonds": rotatable,
                "tpsa": round(
                    tpsa, 2
                ),
                "num_atoms": atoms,
                "num_rings": rings
            },

            "lipinski": {
                "violations": violations,
                "drug_like": violations <= 1
            },

            "predicted_logS": round(
                float(prediction), 4
            ),

            "similar_molecules": similar
        }
```

```
    aspirin = "CC(=O)OC1=CC=CC=C1C(=O)O"

    analysis = molecule_analyzer(
```

```
    aspirin,
    top_n=5
)

analysis
```

```
[01:30:12] DEPRECATION WARNING: please use MorganGenerator
[01:30:12] DEPRECATION WARNING: please use MorganGenerator
{'valid': True,
 'molecular_properties': {'molecular_weight': 180.16,
  'logp': 1.31,
  'h_bond_donors': 1,
  'h_bond_acceptors': 3,
  'rotatable_bonds': 2,
  'tpsa': 63.6,
  'num_atoms': 13,
  'num_rings': 1},
 'lipinski': {'violations': 0, 'drug_like': True},
 'predicted_logS': -2.0009,
 'similar_molecules':           compound_id                    smiles  similarity  measured_logS
 0  Dimethyl phthalate    COC(=O)c1ccccc1C(=O)OC      0.4000         -1.660
 1        Acetophenone            CC(=O)c1ccccc1      0.3793         -1.280
 2          Isoprocarb     CNC(=O)Oc1ccccc1C(C)C      0.3684         -2.863
 3  Diethyl phthalate   CCOC(=O)c1ccccc1C(=O)OCC      0.3636         -2.350
 4        Benzophenone     O=C(c1ccccc1)c2ccccc2      0.3571         -3.120}
```

```
print("🧬 MOLECULE ANALYSIS")
print("=" * 40)

print("\nMolecular Properties")

for key, value in analysis["molecular_properties"].items():
    print(f"{key}: {value}")

print("\nLipinski Analysis")

for key, value in analysis["lipinski"].items():
    print(f"{key}: {value}")

print("\nPredicted Solubility")
print(
    "LogS:",
```

```
    analysis["predicted_logS"]
)

print("\nTop Similar Molecules")

display(
    analysis["similar_molecules"]
)
```

```
🧬 MOLECULE ANALYSIS
========================================
```

```
import joblib

joblib.dump(
    xgb_combined,
    "drug_solubility_xgb.pkl"
)

print("Model saved successfully.")
```

```
num_rings: 1
Model saved successfully.
Lipinski Analysis
```

```
model_config = {
    "radius": 2,
    "n_bits": 2048,
    "feature_count": 2056,
    "target": "measured log solubility in mols per litre"
}

joblib.dump(
    model_config,
    "model_config.pkl"
)
```